

ENGINEERING POST-MORTEM

TicketWave

System Challenges, Root-Cause Diagnoses, and Proven Resolutions

CORE INSIGHT

The Anatomy of Hard Problems

Nearly every expensive defect produced zero exceptions, zero log warnings, and zero test failures. The system simply did the wrong thing while reporting a green build.

Silent Defects in Code Execution



Bypassed Auth Chain

Spring Boot auto-registered `JwtAuthenticationFilter` globally alongside `addFilterBefore`, silently discarding authentication tokens.

Fix: Disabled redundant registration; applied `FilterRegistrationBean` pattern to prevent recurrence.



Detached Entity Updates

With `open-in-view: false`, 3DS confirmation updated a detached entity. API answered `SUCCEEDED` while DB stayed `PENDING_3DS`.

Fix: Added explicit re-`save` and dedicated regression verification test.



Silent JPA Startup Failure

Missing `@Primary` on `DataSource` routing caused `entityManagerFactory` creation to fail with misleading missing-bean errors.

Fix: Explicitly marked primary datasource configuration.

False Testing Guarantees

Skipped IT Suite & Shared State

Issue: Surefire omitted `*IT` files during `mvn verify` while reporting green. Enabling Failsafe immediately broke 8 setup fixtures due to hardcoded DB identifiers.

Fix: Bound Failsafe correctly and moved unique ID generation inside the `newUser` factory, achieving 100% deterministic re-runs.

Flawed Coverage Metrics

Issue: JaCoCo checked unit tests before integration runs, ignored branch thresholds, and let generated MapStruct mappers inflate overall metrics.

Fix: Merged test execution data, excluded generated files, and raised true branch coverage from **74.28%** → **98.05%**.

↕ Concurrency & Race Conditions

Concurrency Hazard	Failure Scenario	Engineering Resolution
Double Refund Race	Two concurrent refund calls both read <code>CONFIRMED</code> and issue double refunds.	Added <code>@Version</code> optimistic lock on <code>Booking</code> entity mapped to <code>409 CONCURRENT_UPDATE</code> .
Seat Lock Deadlocks	Bookings racing for overlapping seats deadlocked on inter-row lock order.	Enforced strict ascending-ID sorting prior to lock acquisition, eliminating lock cycles.
Operator Collision	Operator editing seat attributes clobbered customer's active checkout.	Applied <code>findByIdForUpdate</code> row locks and blocked edits on <code>BOOKED</code> or <code>HELD</code> seats.

Financial Risk Prevention

- **Premature Cancellation Protection:** Refund requests keep bookings `CONFIRMED` and retain seats until approval. Eliminates cases where rejected refunds left customers with neither trip nor money.
- **Isolating Ledger Blips:** Failed ledger writes catch and log at `ERROR` without rolling back valid payments, avoiding 500 errors on successful charges.
- **Defensive Pricing Floor:** Implemented `MIN_DEMAND_MULTIPLIER = 0.10` to prevent misconfigured properties from driving fares to zero or negative.
- **Strict 100% Coverage Mandate:** Enforced complete line and branch coverage on core pricing, hold, and refund policies.

 Software developer debugging code

CI/CD POST-MORTEM

The 2026-08-03 Pipeline Crisis

A stark case study: Every defect was completely invisible on local developer machines with warm caches, existing databases, and fast local networks.

⚡ Cold CI Runner Defects

Four Hidden Environments Hazards

Maven ClassCastException: Cold plugin download polluted classpath. Fixed via `-Dmaven.resolver.transport=wagon` in `.mvn/maven.config`.

E2E Probe Poisoning: Readiness check queried unseeded `GET /api/schedules/1`, caching an empty miss. Switched probe to `/v3/api-docs`.

Docker Engine 26+ Rejection: Testcontainers sent client version 1.32. Passed explicit `docker.api.version=1.44` to Failsafe JVM.

Infinite Watch Loops: Inverted `npm test` flags to ensure CI runners exit cleanly without waiting for interactive input.



💡 Effective Problem-Solving Habits



1. Reproduce Before Fixing

Never patch based on hypotheses alone. Isolate issues in clean, local environments with cold caches before writing fixes.



2. Fix Diagnostics First

When error messages are ambiguous (e.g. `element not found`), improve log output and assertion granularity to pinpoint the exact root cause.



3. Document Refutations

Record disproved hypotheses directly in codebase documentation so future maintainers avoid re-running identical failed experiments.

Refuted Hypotheses Log

Initial Hypothesis	How It Was Refuted & Disproved
Maven failure caused by runner's old Maven version	Pinned Maven 3.9.16 via wrapper; failure reproduced identically.
Contract plugin <code>true<</code> setting at fault	Removed flag; cold build failed unchanged (issue was in execution realm).
E2E failure driven by date/timezone mismatch	Inspection proved spec form defaults to empty date inputs.
E2E failure caused by API rate limiting	Suite passed 5/5 locally with rate limit set to a strict 15 req/min.
Testcontainers fundamentally incompatible with host	Narrowed log filter had hidden the single line identifying API version mismatch.

TicketWave Engineering Retrospective

"A developer machine lies — CI is the ultimate source of truth."

Real Database Testing • Cold Runner CI Validation • 100% Critical Branch Coverage

Image Sources



https://images.stockcake.com/public/a/7/9/a79ea703-9151-49b6-adca-f2b4abe15155_large/terminal-debug-output-stockcake.jpg

Source: stockcake.com



https://png.pngtree.com/thumb_back/fw800/background/20251028/pngtree-server-room-data-flow-technicians-amidst-glowing-cables-and-rack-units-image_20076886.webp

Source: pngtree.com